

PUC Minas

Curso de Engenharia de Software

Disciplina: Sistemas Operacionais

Trabalho Prático 2

Gerenciador de Memória Virtual

Aluno: Pedro Lucas Soares Rezende

Professor: Lucas Bragança da Silva

Belo Horizonte

2026

Sumário

- 1 Introdução
- 2 Fundamentação Teórica
 - 2.1 Memória Virtual
 - 2.2 Paginação
 - 2.3 Tabela de Páginas
 - 2.4 Translation Lookaside Buffer
 - 2.5 Falta de Página
 - 2.6 Algoritmo LRU
 - 2.7 Algoritmo Aging
- 3 Arquitetura da Solução
- 4 Estruturas de Dados Utilizadas
- 5 Desenvolvimento e Implementação
 - 5.1 Tradução de Endereços
 - 5.2 Consulta ao TLB
 - 5.3 Tabela de Páginas
 - 5.4 Tratamento de Faltas de Página
 - 5.5 Substituição de Páginas com Aging
 - 5.6 Atualização dos Contadores de Envelhecimento
 - 5.7 Estatísticas
- 6 Testes Realizados
 - 6.1 Validação de Casos Específicos
- 7 Resultados Obtidos
- 8 Discussão dos Resultados
 - 8.1 Limitações e Considerações de Projeto
- 9 Uso de Inteligência Artificial
- 10 Repositório GitHub
- 11 Conclusão
- 12 Referências Bibliográficas

1 Introdução

A gerência de memória é uma das funções mais importantes de um sistema operacional, pois permite controlar a utilização da memória principal e organizar a forma como os processos acessam seus dados durante a execução. Em sistemas modernos, o uso da memória virtual é essencial para fornecer aos programas um espaço de endereçamento lógico independente da quantidade real de memória física disponível.

Neste trabalho foi desenvolvido um simulador de gerenciamento de memória virtual em linguagem C. O programa tem como objetivo traduzir endereços lógicos para endereços físicos utilizando estruturas clássicas de sistemas operacionais, como tabela de páginas, TLB, paginação por demanda e tratamento de faltas de página.

O projeto simula um espaço de endereçamento virtual de 65.536 bytes, dividido em páginas de 256 bytes. A memória física possui 128 quadros, também com 256 bytes cada, totalizando 32.768 bytes. Como a memória física é menor que o espaço virtual, foi necessário implementar uma política de substituição de páginas. Para isso, foi utilizado o algoritmo Aging, que aproxima o comportamento do algoritmo LRU.

Além disso, foi implementado um TLB com 16 entradas, utilizando a política FIFO para substituição. Ao final da execução, o programa apresenta estatísticas importantes, como a taxa de faltas de página e a taxa de acertos no TLB.

2 Fundamentação Teórica

2.1 Memória Virtual

A memória virtual é uma técnica utilizada pelos sistemas operacionais para permitir que os processos utilizem um espaço de endereçamento maior do que a memória física disponível. Essa abordagem separa o endereço lógico, utilizado pelo programa, do endereço físico, utilizado pela memória principal.

Com isso, o sistema operacional consegue carregar apenas as partes necessárias de um programa na memória física, mantendo outras partes em armazenamento secundário. Essa técnica melhora o aproveitamento da memória, facilita o isolamento entre processos e permite maior flexibilidade na execução de programas.

2.2 Paginação

A paginação é uma técnica de gerenciamento de memória em que o espaço de endereçamento virtual é dividido em blocos de tamanho fixo, chamados páginas. A memória física, por sua vez, é dividida em blocos de mesmo tamanho, chamados quadros.

Neste projeto, cada página possui 256 bytes. Como o endereço lógico considerado possui 16 bits, os 8 bits mais significativos indicam o número da página e os 8 bits menos

significativos indicam o deslocamento dentro da página.

A partir dessa divisão, o sistema identifica em qual quadro físico a página está armazenada e calcula o endereço físico correspondente.

2.3 Tabela de Páginas

A tabela de páginas é a estrutura responsável por armazenar o mapeamento entre páginas virtuais e quadros físicos. Cada entrada da tabela indica se uma página está carregada na memória física e, caso esteja, em qual quadro ela se encontra.

Neste trabalho, a tabela de páginas possui 256 entradas. Cada entrada armazena informações como o número do quadro, bit de validade, bit de referência e contador de envelhecimento utilizado pelo algoritmo Aging.

Quando uma página não está carregada na memória física, sua entrada é considerada inválida. Nesse caso, ocorre uma falta de página.

2.4 Translation Lookaside Buffer

O Translation Lookaside Buffer, conhecido como TLB, é uma pequena memória cache utilizada para armazenar traduções recentes entre páginas virtuais e quadros físicos. Seu objetivo é reduzir a quantidade de acessos à tabela de páginas.

Neste projeto, o TLB possui 16 entradas. Quando o endereço lógico é lido, o simulador primeiro consulta o TLB. Se a página estiver presente, ocorre um TLB Hit, e o quadro físico é obtido rapidamente. Caso contrário, ocorre um TLB Miss, e a tabela de páginas precisa ser consultada.

Para substituição de entradas no TLB, foi utilizada a política FIFO. Assim, quando o TLB está cheio, a entrada mais antiga é substituída pela nova tradução.

2.5 Falta de Página

Uma falta de página ocorre quando o programa tenta acessar uma página que não está carregada na memória física. Nesse caso, o sistema precisa buscar a página no armazenamento secundário e carregá-la para a memória principal.

No projeto, o armazenamento secundário é representado pelo arquivo `BACKING_STORE.bin`. Quando ocorre uma falta de página, o programa usa `fseek` para posicionar a leitura na página correta e `fread` para copiar os 256 bytes correspondentes para um quadro da memória física.

2.6 Algoritmo LRU

O algoritmo LRU, sigla para Least Recently Used, remove da memória a página que não é utilizada há mais tempo. A ideia é que páginas usadas recentemente tenham

maior chance de serem acessadas novamente em breve.

Embora seja eficiente, o LRU puro pode ser custoso de implementar, pois exige manter um histórico preciso dos acessos às páginas. Por isso, muitos sistemas utilizam aproximações do LRU.

2.7 Algoritmo Aging

O algoritmo Aging é uma aproximação do LRU. Ele utiliza um contador de envelhecimento para cada página residente na memória. Cada página também possui um bit de referência, que indica se ela foi acessada recentemente.

Periodicamente, o contador de cada página é deslocado uma posição para a direita. Em seguida, o bit de referência é inserido no bit mais significativo do contador. Depois disso, o bit de referência é zerado.

Quando uma página precisa ser removida, o sistema escolhe aquela que possui o menor contador de envelhecimento. Dessa forma, páginas pouco acessadas tendem a apresentar valores menores e são escolhidas como vítimas.

3 Arquitetura da Solução

A solução foi organizada em módulos, separando as responsabilidades do simulador. Essa divisão facilita a leitura, a manutenção e o entendimento do código.

A estrutura principal do projeto ficou organizada da seguinte forma:

- `main.c`: controla o fluxo principal do programa;
- `tlb.c`: implementa o TLB e a política FIFO;
- `page_table.c`: implementa a tabela de páginas e os dados do Aging;
- `memory.c`: gerencia a memória física e o tratamento de faltas de página;
- `statistics.c`: calcula e imprime as estatísticas finais;
- `config.h`: define as constantes principais do projeto.

Essa divisão modular permitiu implementar cada parte do simulador separadamente, mantendo o código mais organizado e próximo da estrutura de um sistema operacional real.

4 Estruturas de Dados Utilizadas

A implementação utiliza vetores e estruturas simples em C para representar os principais componentes da memória virtual.

A memória física foi representada por uma matriz de caracteres sinalizados, onde cada linha representa um quadro físico e cada coluna representa um byte dentro do quadro.

A tabela de páginas foi representada por um vetor de entradas. Cada entrada possui:

- número do quadro físico;
- bit de validade;
- bit de referência;
- contador de envelhecimento de 8 bits.

O TLB também foi representado por um vetor de entradas. Cada entrada possui:

- número da página;
- número do quadro;
- bit de validade.

Além disso, foi utilizado um vetor auxiliar para indicar qual página está carregada em cada quadro físico. Esse vetor é importante durante a substituição de páginas, pois permite localizar a página associada ao quadro escolhido.

5 Desenvolvimento e Implementação

5.1 Tradução de Endereços

O programa lê endereços lógicos inteiros a partir da entrada padrão. Como o projeto considera apenas endereços de 16 bits, é aplicada uma máscara sobre o endereço lido:

```
logical_address = input_address & 0xFFFF
```

Após isso, o número da página é obtido deslocando o endereço 8 bits para a direita. O offset é obtido utilizando os 8 bits menos significativos.

Esse processo permite separar corretamente o endereço lógico em página e deslocamento, conforme a estrutura definida no enunciado.

5.2 Consulta ao TLB

Após extrair o número da página, o programa consulta o TLB. Se a página for encontrada, o quadro físico correspondente é retornado imediatamente, caracterizando um TLB Hit.

Caso a página não seja encontrada no TLB, ocorre um TLB Miss. Nessa situação, o programa consulta a tabela de páginas para verificar se a página está carregada na memória física.

5.3 Tabela de Páginas

A tabela de páginas é inicializada com todas as entradas inválidas. Quando uma página é carregada na memória, sua entrada é atualizada com o número do quadro físico correspondente.

Também são atualizados o bit de validade, o bit de referência e o contador de envelhecimento. Quando uma página é removida da memória, sua entrada é invalidada para impedir que acessos futuros utilizem um mapeamento incorreto.

5.4 Tratamento de Faltas de Página

Quando a página não está presente na tabela de páginas, ocorre uma falta de página. O programa então verifica se existe algum quadro livre na memória física.

Se houver quadro disponível, a página é carregada diretamente nele. Caso contrário, é necessário selecionar uma página vítima utilizando o algoritmo Aging.

Depois de escolhido o quadro, o programa lê a página correspondente no arquivo `BACKING_STORE.bin`. A leitura é realizada considerando que cada página possui 256 bytes. Assim, o deslocamento dentro do arquivo é calculado multiplicando o número da página pelo tamanho da página.

Após a leitura, a tabela de páginas é atualizada e a nova tradução é inserida no TLB.

5.5 Substituição de Páginas com Aging

Quando não existem quadros livres, o simulador precisa remover uma página da memória física. Para isso, percorre a tabela de páginas em busca da página válida com menor contador de envelhecimento.

A página com menor contador é escolhida como vítima, pois representa aquela que foi menos utilizada recentemente. Ao removê-la, sua entrada na tabela de páginas é invalidada e sua eventual entrada no TLB também é removida.

Esse cuidado é importante para evitar inconsistências, pois uma tradução antiga no TLB poderia apontar para um quadro que passou a armazenar outra página.

5.6 Atualização dos Contadores de Envelhecimento

A cada acesso à memória, o bit de referência da página acessada é marcado. Em seguida, os contadores de envelhecimento são atualizados.

Cada contador é deslocado uma posição para a direita. Se o bit de referência estiver marcado, o bit mais significativo do contador recebe 1. Depois disso, o bit de referência é zerado.

Esse processo permite manter uma aproximação do histórico de acessos às páginas, favorecendo a permanência de páginas acessadas recentemente.

5.7 Estatísticas

Ao final da execução, o simulador apresenta estatísticas sobre o desempenho da tradução de endereços. São exibidos:

- número de endereços traduzidos;
- quantidade de faltas de página;
- taxa de faltas de página;
- quantidade de acertos no TLB;
- taxa de acertos no TLB.

Essas estatísticas ajudam a avaliar o comportamento do simulador e a influência da localidade de referência nos acessos à memória.

6 Testes Realizados

Os testes foram realizados utilizando os arquivos de entrada fornecidos pelo projeto: `addresses_random.txt` e `addresses_location.txt`.

O arquivo com endereços aleatórios foi utilizado para avaliar o comportamento do simulador em acessos menos previsíveis. Já o arquivo com localidade de referência foi utilizado para observar o impacto de acessos mais concentrados em determinadas regiões da memória.

Durante os testes, foram avaliados os seguintes pontos:

- extração correta do número da página e do offset;
- funcionamento da busca no TLB;
- consulta e atualização da tabela de páginas;

- carregamento de páginas a partir do `BACKING_STORE.bin`;
- tratamento de faltas de página;
- substituição de páginas pelo algoritmo Aging;
- invalidação de entradas antigas da tabela de páginas e do TLB;
- geração das estatísticas finais.

6.1 Validação de Casos Específicos

Para garantir a corretude da implementação, foram realizados testes de validação com casos específicos durante a execução do simulador.

Teste 1: Extração de Página e Offset

Endereço lógico de entrada: 14592. Aplicando a máscara de 16 bits, obtém-se página 57 e offset 0. O endereço físico calculado é 0, e o valor lido da memória é 0. Este teste valida que a decomposição do endereço lógico em página e offset está funcionando corretamente conforme a estrutura de 16 bits especificada.

Teste 2: Page Fault e Carregamento do `BACKING_STORE.bin`

Endereço lógico: 3278 (primeira execução). Esperava-se um page fault, pois a página não estava carregada. O programa leu corretamente a página 12 do arquivo `BACKING_STORE.bin` e obteve o valor -50. Este teste confirma que o tratamento de falta de página está funcionando, incluindo o posicionamento correto no arquivo de armazenamento secundário e a leitura dos 256 bytes da página.

Teste 3: Algoritmo Aging e Substituição de Páginas

Após preencher os 128 quadros físicos disponíveis, a próxima página que causa falta deve substituir a página com menor contador de envelhecimento. Na execução com `addresses_random.txt`, foram registrados 4958 page faults em 10.000 acessos, resultando em uma taxa de 49,6%. Esta proporção é consistente com um padrão aleatório onde a memória física é significativamente menor que o espaço virtual, confirmando que a substituição ocorreu corretamente quando a memória ficou cheia.

Teste 4: TLB com Política FIFO

Nos primeiros 16 acessos a páginas diferentes, todas as entradas do TLB devem estar vazias. Após o 17º acesso a uma página diferente, a entrada mais antiga deve ser substituída. No teste aleatório, obteve-se 654 TLB Hits em 10.000 acessos (6,5%), indicando baixa reutilização. No teste com localidade, obteve-se 7925 TLB Hits em 10.000 acessos (79,2%), indicando alta reutilização. Este resultado valida que a política FIFO está funcionando corretamente.

7 Resultados Obtidos

Os testes foram realizados utilizando os dois arquivos de entrada fornecidos pelo projeto: `addresses_random.txt` e `addresses_location.txt`. Em ambos os casos foram processados 10.000 endereços lógicos.

A Tabela 1 apresenta os resultados obtidos durante a execução do simulador.

Tabela 1: Resultados dos testes realizados

Métrica	Aleatório	Localidade
Endereços Traduzidos	10000	10000
Page Faults	4958	1164
Page Fault Rate	0.496	0.116
TLB Hits	654	7925
TLB Hit Rate	0.065	0.792

Os resultados demonstram diferenças significativas entre os dois padrões de acesso utilizados nos testes.

No arquivo `addresses_random.txt`, que utiliza endereços distribuídos de forma aleatória, foram registradas 4958 faltas de página, correspondendo a uma taxa de Page Fault de 49,6%. Além disso, foram observados apenas 654 acertos no TLB, resultando em uma taxa de acerto de aproximadamente 6,5%.

Já no arquivo `addresses_location.txt`, que apresenta localidade de referência, o comportamento foi consideravelmente melhor. Foram registradas apenas 1164 faltas de página, representando uma taxa de 11,6%. O número de acertos no TLB aumentou para 7925, resultando em uma taxa de acerto de aproximadamente 79,2%.

Esses resultados confirmam a importância da localidade de referência para o desempenho de sistemas de memória virtual. Quando os acessos tendem a ocorrer em regiões próximas da memória, as páginas carregadas recentemente possuem maior probabilidade de serem reutilizadas, reduzindo a ocorrência de faltas de página e aumentando a eficiência do TLB.

Os testes também demonstraram o funcionamento adequado do algoritmo Aging, que foi capaz de selecionar páginas pouco utilizadas para substituição quando a memória física atingiu sua capacidade máxima.

8 Discussão dos Resultados

A comparação entre os dois arquivos de entrada evidencia claramente o impacto da localidade de referência sobre o desempenho da memória virtual.

No cenário de acessos aleatórios, a taxa de faltas de página atingiu 49,6%, indicando que quase metade dos acessos exigiu carregamento de páginas a partir do armazenamento

secundário. Da mesma forma, a taxa de acerto do TLB foi de apenas 6,5%, demonstrando baixa reutilização das traduções armazenadas.

Por outro lado, o cenário com localidade de referência apresentou resultados significativamente superiores. A taxa de faltas de página caiu para 11,6%, enquanto a taxa de acerto do TLB alcançou 79,2%. Isso ocorre porque acessos próximos tendem a reutilizar páginas e traduções recentemente utilizadas, aumentando a eficiência tanto da tabela de páginas quanto do TLB.

Os resultados obtidos estão de acordo com os princípios teóricos estudados na disciplina de Sistemas Operacionais. A presença de localidade espacial e temporal favorece o reaproveitamento das estruturas de memória, reduzindo o custo das traduções e minimizando operações de carregamento de páginas.

Além disso, o algoritmo Aging apresentou comportamento consistente durante os testes, realizando a substituição de páginas de forma eficiente e mantendo a coerência entre memória física, tabela de páginas e TLB.

8.1 Limitações e Considerações de Projeto

A implementação atual apresenta características que refletem decisões de projeto balanceando realismo com praticidade.

Atualização do Contador de Envelhecimento

O algoritmo Aging implementado atualiza o contador de todas as páginas válidas a cada acesso à memória. Embora funcional e correto do ponto de vista algorítmico, esta abordagem é mais agressiva que a típica em sistemas operacionais reais, onde a atualização ocorre em intervalos periódicos fixos (a cada 100 ou 1000 ciclos de relógio) ou através de interrupções de timer. A implementação atual sacrifica realismo em favor da simplicidade computacional, mas mantém a corretude do algoritmo e permite validar seu comportamento de forma determinística.

Política FIFO do TLB

A implementação do TLB não move entradas já existentes para o final da fila quando são acessadas novamente. Se uma página já está no TLB e é acessada uma segunda vez, sua entrada permanece na mesma posição. Isso é uma variação válida de FIFO que melhora a eficiência, mas difere da implementação clássica de FIFO puro. A escolha foi mantida para simplificar a implementação sem comprometer a semântica de cache.

Impacto da Localidade de Referência

Os resultados experimentais demonstram que o princípio da localidade de referência é fundamental para o desempenho de sistemas com memória virtual. A melhoria de aproximadamente 12 vezes na taxa de TLB Hit (de 6,5% para 79,2%) e a redução de 4,3 vezes na taxa de Page Fault (de 49,6% para 11,6%) ilustram por que programas reais se beneficiam significativamente de caches e TLBs. Este resultado reforça a importância de

projetar estruturas de dados e algoritmos que exploram a localidade.

Coerência entre Estruturas de Dados

Um aspecto crítico da implementação foi garantir a coerência entre as três estruturas principais: memória física, tabela de páginas e TLB. Quando uma página é removida da memória, sua entrada na tabela de páginas é invalidada e sua entrada no TLB (se existir) também é removida. Isso previne o erro de manter traduções obsoletas em cache, que poderia levar a acessos incorretos à memória. Os testes confirmaram que essa coerência foi mantida durante toda a execução.

9 Uso de Inteligência Artificial

Durante o desenvolvimento do trabalho, ferramentas de Inteligência Artificial foram utilizadas como apoio para revisão de conceitos, esclarecimento de dúvidas, organização da documentação e conferência da estrutura do relatório.

O uso ocorreu de forma auxiliar, especialmente para revisar explicações sobre memória virtual, paginação, TLB, faltas de página e algoritmo Aging. Também houve apoio na revisão da clareza do texto e na organização das seções do relatório.

Os prompts utilizados seguiram uma abordagem baseada em especificação, descrevendo claramente os requisitos do projeto, como tamanho da página, quantidade de quadros, tamanho do TLB, política FIFO, tratamento de faltas de página e substituição por Aging.

Exemplos de prompts utilizados:

- Explicar o funcionamento do algoritmo Aging como aproximação do LRU.
- Revisar a estrutura de um relatório técnico de Sistemas Operacionais.
- Descrever o funcionamento de uma TLB utilizando política FIFO.
- Organizar um relatório técnico em LaTeX para o projeto de memória virtual.

As respostas obtidas foram utilizadas apenas como material de apoio, sendo posteriormente verificadas e ajustadas conforme os requisitos do projeto. A responsabilidade técnica pelo código, pela validação da implementação e pela compreensão das decisões adotadas permaneceu sob responsabilidade do aluno.

10 Repositório GitHub

O projeto completo está disponível no seguinte repositório público:

<https://github.com/pedrolsrt/tp2-gerenciador-memoria-virtual>

O repositório contém a estrutura completa do projeto, incluindo arquivos-fonte, cabeçalhos, Makefile, arquivos de dados, documentação e relatório técnico.

11 Conclusão

O desenvolvimento deste trabalho permitiu aplicar, de forma prática, conceitos importantes de Sistemas Operacionais relacionados à gerência de memória virtual.

Foram implementados os principais componentes envolvidos na tradução de endereços, incluindo TLB, tabela de páginas, memória física, paginação por demanda e tratamento de faltas de página. Além disso, foi implementada uma política de substituição baseada no algoritmo Aging, aproximando o comportamento do LRU.

A realização dos testes permitiu observar a diferença entre acessos aleatórios e acessos com localidade de referência. Também foi possível compreender a importância do TLB para melhorar o desempenho da tradução de endereços e a necessidade de manter as estruturas de dados sempre consistentes após substituições de páginas.

De modo geral, o projeto contribuiu para uma compreensão mais concreta do funcionamento interno da memória virtual em sistemas operacionais, reforçando conceitos teóricos por meio de uma implementação prática em linguagem C.

12 Referências Bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert. *Modern Operating Systems*. 4. ed. Pearson, 2015.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. *Operating System Concepts*. 10. ed. Wiley, 2018.

STALLINGS, William. *Operating Systems: Internals and Design Principles*. 9. ed. Pearson, 2018.